

ReAct 模式拆解：Agent 如何做到"边想边做"

原创 James的成长日记 James的成长日记 2026年4月9日 16:36 广东

大家好，我是James。

上一篇我们从零实现了一个 Tool-Calling Agent，用 LangChain.js 把「工具调用」跑通了。但你有没有发现，Agent 调用工具的时候，它怎么知道该用哪个工具？调了之后发现结果不对，它又怎么调整？

你让 Agent 查一个问题，它直接给了你答案，但答案是错的。你不知道它「想了什么」，也不知道它「为什么这么做」。

这不是模型变笨了，是架构的问题。**ReAct 模式就是解决这个问题**——让 Agent 把「思考过程」和「行动步骤」显式地交织在一起，像人解题一样，边想边做，随时调整。

一句话定义

ReAct = Reasoning（推理）+ Acting（行动）的交织循环。

类比：就像一个侦探破案。他不会一开始就下结论，而是先猜测（Reasoning），然后去现场取证（Acting），根据证据更新判断，再继续行动，直到案子破了。

普通 LLM 是：看一眼 → 直接给答案。ReAct Agent 是：想一步 → 做一步 → 看结果 → 再想 → 再做.....

为什么 ReAct 比普通工具调用强

先看一个对比：

没有 ReAct 的 Agent 接到任务「帮我查今天北京天气，并告诉我要不要带伞」：

1. 调用 `getWeather("北京")`
2. 返回结果

有 ReAct 的 Agent 处理同样任务：

Thought: 用户问北京今天天气和是否需要带伞，我需要先查天气数据。
Action: `getWeather({"city": "北京"})`
Observation: 今天北京 多云转阵雨，最高气温 18°C，降雨概率 75%

Thought: 降雨概率 75%，属于高概率降雨，建议带伞。可以给出结论了。
Final Answer: 今天北京有阵雨，降雨概率 75%，建议带伞出门。

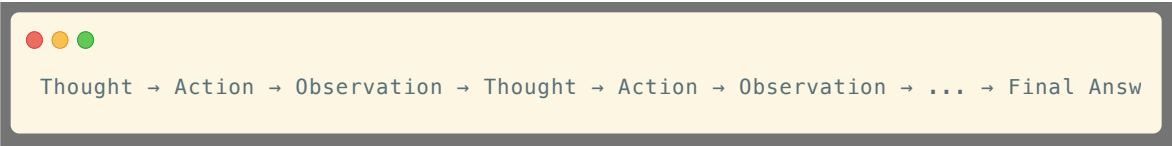
区别在哪？中间多了 Thought 和 Observation 两个步骤。

这带来两个好处：

- 1. 可解释：你能看到 Agent 「为什么这么做」
- 2. 可纠错：如果工具返回的结果不对，Thought 阶段会意识到，并调整下一步行动

ReAct 的核心循环

ReAct 的执行流是一个固定的循环：



每一轮包含三件事：

步骤	角色	内容
Thought	LLM 自己写	当前推理，下一步打算
Action	LLM 决定，代码执行	调用哪个工具，参数是什么
Observation	工具返回，注入上下文	工具执行结果

这个循环结束的条件：LLM 输出 Final Answer:，代表它认为已经有足够信息回答用户了。

从零实现一个 ReAct Agent

我们不用框架封装，手写一个最小 ReAct 循环，搞清楚底层在干什么。

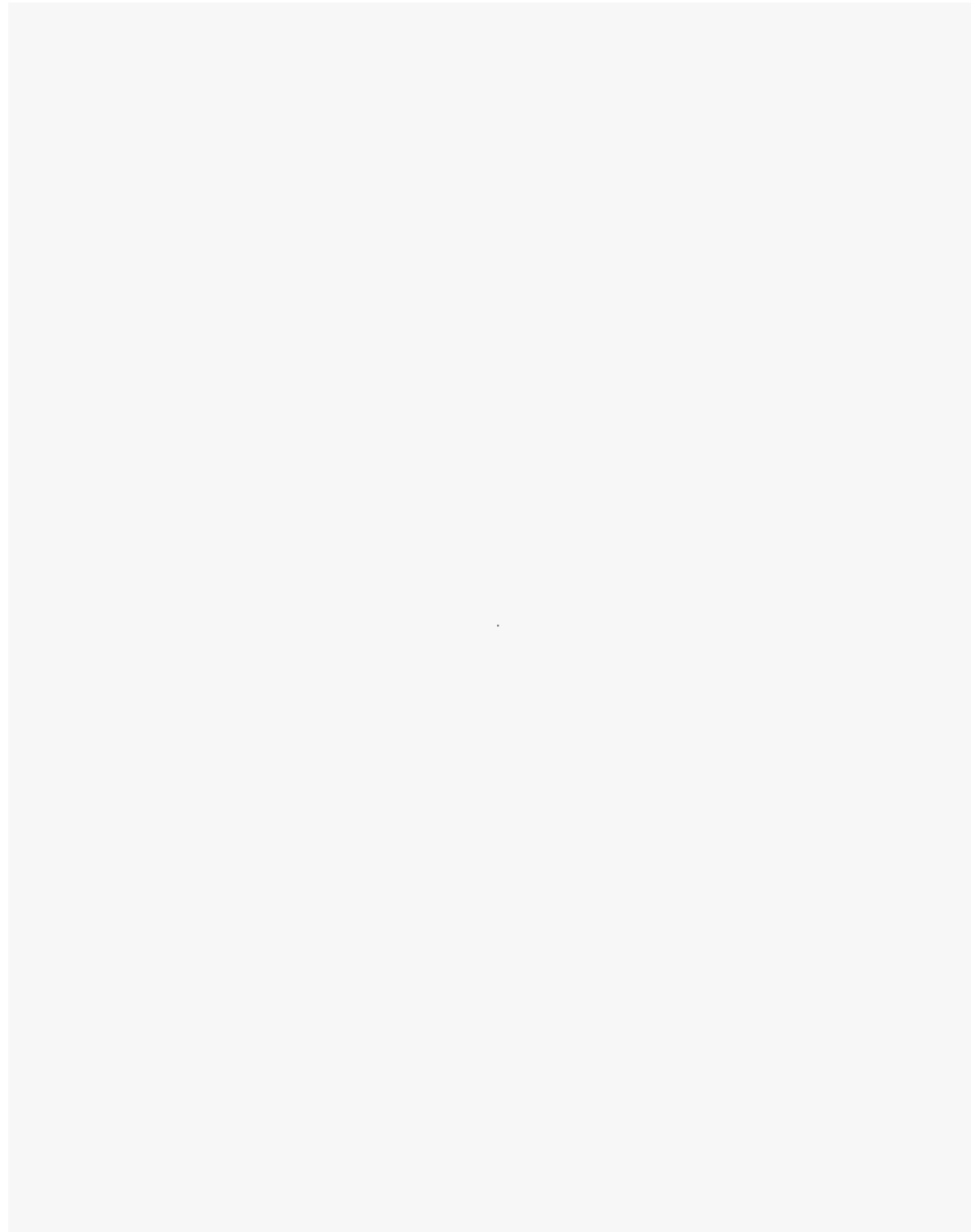
Step 1: 定义工具集

```
// tools.ts
export interface Tool {
  name: string;
  description: string;
  execute: (input: string) => Promise<string>;
}
```

```
// 模拟天气工具
const weatherTool: Tool = {
  name: "getWeather",
  description: "查询城市天气。输入格式: {city: '城市名'}",
  execute: async (input: string) => {
    const { city } = JSON.parse(input);
    // 实际使用中替换为真实 API
    const fakeData: Record<string, string> = {
      "北京": "多云转阵雨, 18°C, 降雨概率75%",
      "上海": "晴, 23°C, 降雨概率10%",
    };
    return fakeData[city] ?? "暂无该城市天气数据";
  }
};

// 模拟计算工具
const calcTool: Tool = {
  name: "calculate",
  description: "执行数学计算。输入格式: {expression: '数学表达式'}",
  execute: async (input: string) => {
    const { expression } = JSON.parse(input);
    // 注意: 生产环境不要用 eval, 用 mathjs 等安全库
    try {
      return String(eval(expression));
    } catch {
      return "计算出错: 表达式无效";
    }
  }
};

export const tools = [weatherTool, calcTool];
```



Step 2: 构建 ReAct System Prompt

```

// react-prompt.ts
import { Tool } from "../tools";

export function buildReActSystemPrompt(tools: Tool[]): string {
  // 把工具列表格式化进 prompt, 让 LLM 知道有哪些工具可用
  const toolDescriptions = tools
    .map(t => `- ${t.name}: ${t.description}`)
    .join("\n");

```

```
return `你是一个会使用工具的 AI 助手。回答问题时严格遵循以下格式：
```

你有以下工具可以使用：

```
${toolDescriptions}
```

回答格式（严格遵守）：

Thought： 你的推理过程，分析当前情况和下一步计划

Action： 工具名称

Action Input： {"参数": "值"}

收到工具结果后继续：

Observation： [工具返回的结果，由系统填入]

Thought： 根据观察结果更新推理

...（可以重复多次 Thought/Action/Observation）

当你有足够信息回答时：

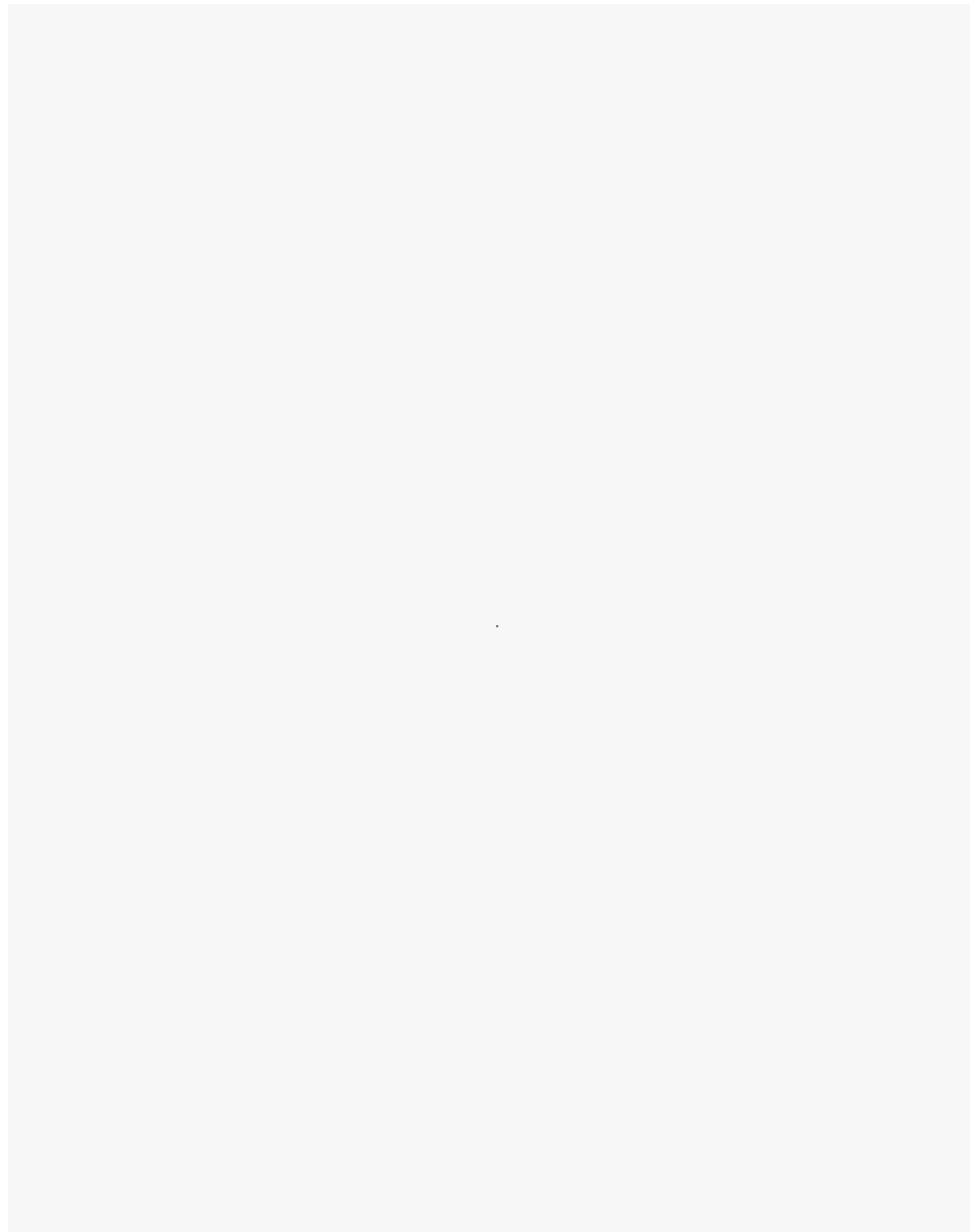
Thought： 已经有足够信息了

Final Answer： 最终回答内容

重要规则：

1. 每次只能调用一个工具
2. Action Input 必须是合法的 JSON
3. 不要编造 Observation，等待系统返回
4. 确定答案后输出 Final Answer，不再调用工具；

```
}
```



Step 3: 实现解析逻辑

```
// react-parser.ts
export interface ReActStep {
  thought: string;
  action?: string;      // 如果有工具调用
  actionInput?: string; // 工具参数
  finalAnswer?: string; // 如果已有结论
}

export function parseReActResponse(text: string): ReActStep {
```

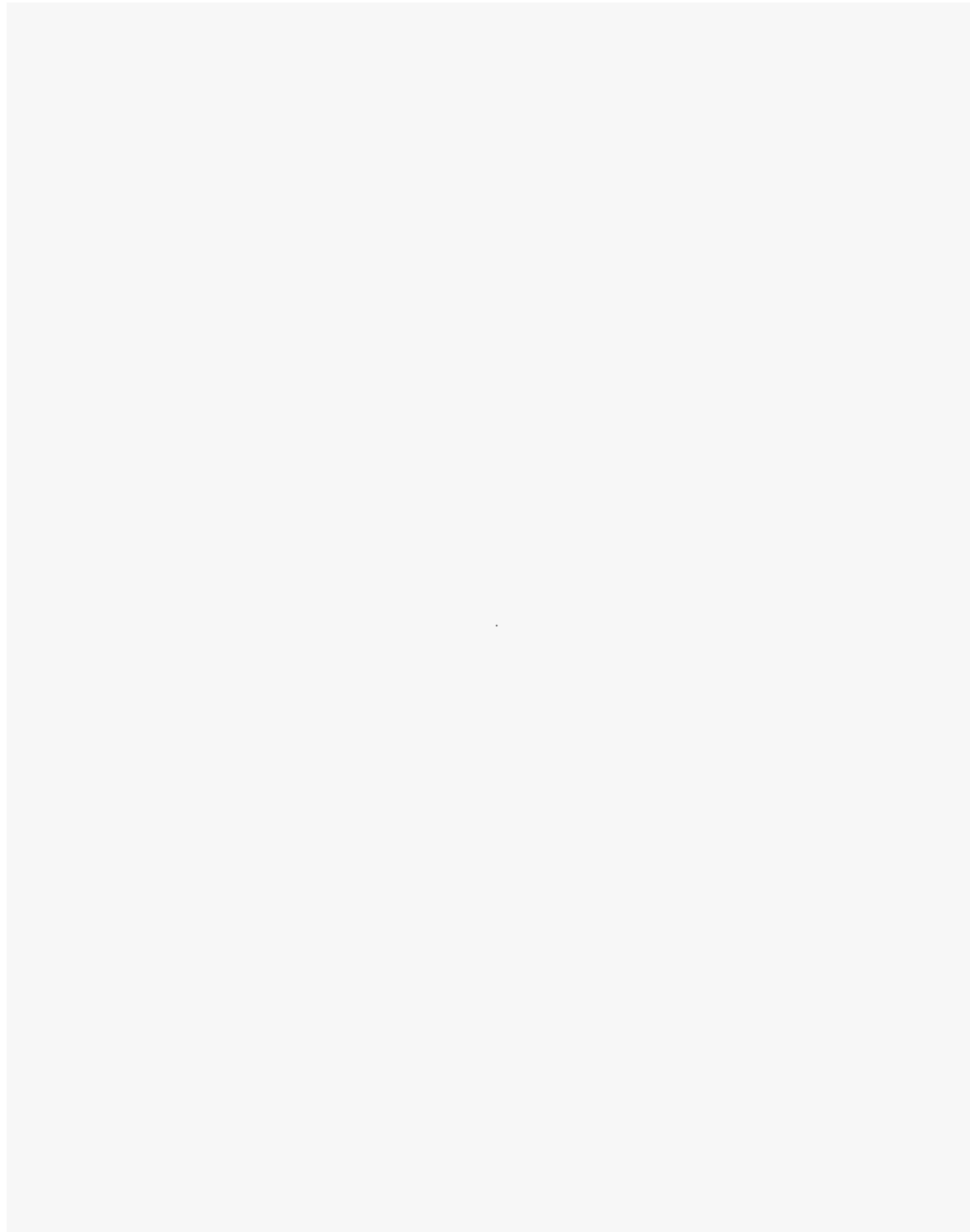
```
const step: ReActStep = { thought: "" };

// 提取 Thought
const thoughtMatch = text.match(/Thought:\s*([\s\S]*?)(?=Action:|Final Answer:|$)
if (thoughtMatch) {
  step.thought = thoughtMatch[1].trim();
}

// 提取 Final Answer (优先判断)
const finalMatch = text.match(/Final Answer:\s*([\s\S]*?)$/);
if (finalMatch) {
  step.finalAnswer = finalMatch[1].trim();
  return step;
}

// 提取 Action
const actionMatch = text.match(/Action:\s*(.+)/);
const inputMatch = text.match(/Action Input:\s*([\s\S]*?)(?=Thought:|Observation)
if (actionMatch) {
  step.action = actionMatch[1].trim();
  step.actionInput = inputMatch ? inputMatch[1].trim() : "{}";
}

return step;
}
```



Step 4: 主循环

```

● ● ●
// react-agent.ts
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage, AIMessage } from "@langchain/core/messages";
import { tools } from "./tools";
import { buildReActSystemPrompt } from "./react-prompt";
import { parseReActResponse } from "./react-parser";

async function runReActAgent(userQuery: string, maxSteps = 10) {
  const llm = new ChatOpenAI({ model: "gpt-4o", temperature: 0 });

```



```

const systemPrompt = buildReActSystemPrompt(tools);

// 对话历史: 记录完整的 Thought/Action/Observation 轨迹
const messages = [
  new SystemMessage(systemPrompt),
  new HumanMessage(userQuery),
];

console.log(`\n👤 用户问题: ${userQuery}`);
console.log("-".repeat(50));

for (let step = 0; step < maxSteps; step++) {
  // 调用 LLM, 获取下一步行动
  const response = await llm.invoke(messages);
  const responseText = response.content as string;

  console.log(`\n[Step ${step + 1}]`);
  console.log(responseText);

  // 解析 LLM 的输出
  const parsed = parseReActResponse(responseText);

  // 如果 LLM 给出了最终答案, 结束循环
  if (parsed.finalAnswer) {
    console.log(`\n✅ 最终答案: `, parsed.finalAnswer);
    return parsed.finalAnswer;
  }

  // 如果 LLM 要调用工具
  if (parsed.action) {
    // 把 LLM 的思考和行动加入历史
    messages.push(new AIMessage(responseText));

    // 找到对应工具并执行
    const tool = tools.find(t => t.name === parsed.action);
    let observation: string;

    if (!tool) {
      observation = `工具 "${parsed.action}" 不存在, 可用工具: ${tools.map(t => t.name).join(", ")}`;
    } else {
      try {
        observation = await tool.execute(parsed.actionInput ?? "{}");
      } catch (e) {
        observation = `工具执行出错: ${e}`;
      }
    }

    console.log(`\nObservation: ${observation}`);

    // 把工具结果作为 Human 消息注入 (模拟 Observation)
  }
}

```

```
messages.push(new HumanMessage(`Observation: ${observation}`));
} else {
  // LLM 没有行动也没有最终答案，说明输出格式不对
  console.warn("⚠️ 解析失败，LLM 未按格式输出");
  break;
}
}

return "达到最大步骤数，未能得出最终答案";
}

// 测试
runReActAgent("今天北京天气怎么样？要带伞吗？");
```

LangChain.js 封装版：5 行搞定

手写版帮你理解原理。生产环境用 LangChain.js 内置的 ReAct Agent：

```
import { createReactAgent } from "@langchain/langgraph/prebuilt";
import { ChatOpenAI } from "@langchain/openai";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

// 用 LangChain tool() 包装工具，带类型校验
const getWeather = tool(
  async ({ city }: { city: string }) => {
    // 替换为真实 API
    return `${city}: 多云转阵雨, 18°C, 降雨概率75%`;
  },
```

```
{
  name: "getWeather",
  description: "查询城市当天天气, 输入城市名称",
  schema: z.object({
    city: z.string().describe("城市名, 如 北京、上海"),
  }),
}

);

const model = new ChatOpenAI({ model: "gpt-4o", temperature: 0 });

// createReactAgent 内部实现了完整的 ReAct 循环
const agent = createReactAgent({
  llm: model,
  tools: [getWeather],
});

// 调用 Agent
const result = await agent.invoke({
  messages: [{ role: "user", content: "今天北京天气怎么样? 要带伞吗? " }],
});

// 最后一条消息就是最终答案
console.log(result.messages.at(-1)?.content);
```

封装版的 ReAct 循环由 LangGraph 状态机驱动, 不需要手动管理消息历史。

流式输出 Thought 过程

默认情况下你等到 `Final Answer` 才能看到输出。用流式可以实时看到 Agent 的思考过程:

```
// 流式输出: 实时看到 Thought → Action → Observation
const stream = await agent.stream({
  messages: [{ role: "user", content: "北京今天要带伞吗? " }],
});

for await (const chunk of stream) {
  // chunk 包含每一步的中间状态
  if (chunk.agent) {
    // LLM 思考阶段的流式 token
    process.stdout.write(chunk.agent.messages[0].content as string);
  }
  if (chunk.tools) {
    // 工具执行结果
    console.log("\n🔧 工具返回: ", chunk.tools.messages[0].content);
  }
}
```

效果: 用户能实时看到 Agent 在「想什么」, 体验接近 ChatGPT 的 reasoning 展示。



常见坑

坑 1：最大步骤数没设，Agent 死循环

```
// ❌ 没有步骤限制，工具报错时 Agent 会无限重试
const agent = createReactAgent({ llm: model, tools });

// ✅ 设置递归限制，超出后抛出可捕获的错误
```

```
const result = await agent.invoke(input, {
  recursionLimit: 10, // 最多 10 步
});
```

LLM 有时候会陷入「工具出错 → 重试 → 还是出错」的循环，不设上限会烧掉大量 token。

坑 2: Observation 太长，撑爆 Context Window

```
// ❌ 直接把工具返回塞进上下文，数据库查询返回 5000 行就爆了
const observation = await dbQueryTool.execute(input);
messages.push(new HumanMessage(`Observation: ${observation}`));

// ✅ 截断 + 摘要：超过阈值时只保留关键信息
const MAX_OBS_LENGTH = 2000;
let observation = await dbQueryTool.execute(input);
if (observation.length > MAX_OBS_LENGTH) {
  observation = observation.slice(0, MAX_OBS_LENGTH) + "\n...(结果已截断, 原始长度: "
}
```

坑 3: 工具描述写得太模糊，LLM 选错工具

```
// ❌ 描述太泛，LLM 不知道什么时候该用
const badTool = tool(fn, {
  name: "search",
  description: "搜索信息",
});

// ✅ 描述精准，包含触发场景和排除场景
const goodTool = tool(fn, {
  name: "webSearch",
  description: "搜索互联网上的实时信息，如新闻、当前价格、最新事件。不适用于查询本地文件或数据库数"
});
```

LLM 选工具靠的是 `description`，写模糊就等于给它出题，答错了别怪模型。

坑 4: Action Input 解析不到，报 JSON 错误

```
// ❌ 直接 JSON.parse, LLM 输出格式稍有偏差就崩
const input = JSON.parse(parsed.actionInput);

// ✅ 容错处理：提取 JSON 子串，处理 LLM 可能带的 markdown 代码块
```

```
function safeParseInput(raw: string): unknown {
  // 去掉 LLM 可能加的 ```json ... ``` 包裹
  const cleaned = raw.replace(/```json\n?/g, "").replace(/```\n?/g, "").trim();
  try {
    return JSON.parse(cleaned);
  } catch {
    // 如果还是解析不了, 返回原始字符串让工具自己处理
    return { input: cleaned };
  }
}
```

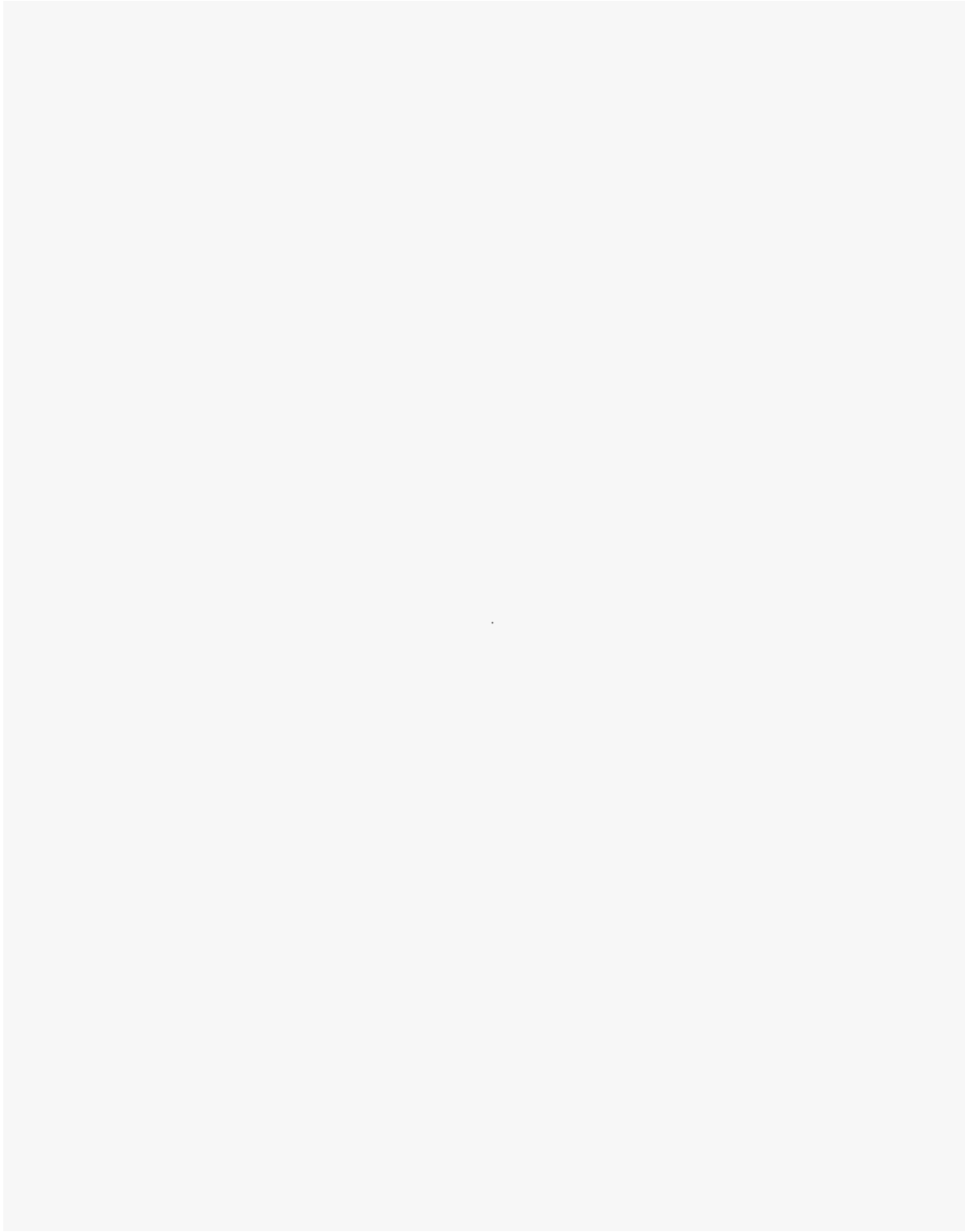
GPT-4o 输出的 Action Input 有时会带 markdown 代码块标记, 裸 `JSON.parse` 会直接报错。

坑 5: Thought 过程不记录, 调试时一头雾水

```
// ❌ 只拿最终答案, 中间过程扔掉
const result = await agent.invoke(input);
console.log(result.messages.at(-1)?.content);

// ✅ 保存完整轨迹, 出错时能回放
const result = await agent.invoke(input);
const trajectory = result.messages.map(m => ({
  role: m.getType(),
  content: m.content,
}));
// 存到日志系统或打印出来
console.log(JSON.stringify(trajectory, null, 2));
```

生产环境的 Agent 一定要记录完整的 ReAct 轨迹, 这是调试的唯一线索。



可收藏清单

ReAct Agent 上线前自查：

- System Prompt 里包含了所有工具的名称和描述
- 设置了 recursionLimit，防止死循环
- Observation 有长度截断，避免撑爆 context
- Action Input 解析有容错，不裸 JSON.parse
- 工具 description 包含触发场景和排除场景
- 生产环境记录完整 ReAct 轨迹（Thought + Action + Observation）

- 测试过工具报错时 Agent 的行为（是重试还是给出错误答案）

总结

这篇我们把 ReAct 从头到尾拆了一遍：

- **ReAct = Reasoning + Acting**，核心在于 Thought 和 Observation 的交织循环，而不是单纯的工具调用
- **普通 Tool-Calling 的问题**是模型一次性决策，遇到复杂任务容易翻车；ReAct 让模型边想边做，步步修正
- **手写 ReAct 循环**帮你理解底层：解析 Thought → 执行 Action → 回传 Observation → 继续思考
- **createReactAgent** 是 LangChain 封装好的生产版本，内置循环控制、错误处理、记忆支持
- **流式输出 Thought**让用户看到 Agent 的推理过程，透明度和信任感同时提升
- **5 个真实坑**：死循环、Context 爆炸、工具描述模糊、JSON 解析容错、轨迹不记录——每一个都是生产事故的源头

ReAct 不是魔法，它只是给了模型一个"允许犯错、允许修正"的框架。理解这个，你才能在 Agent 出错时知道该从哪里下手。

下一篇我们进入 Agent 记忆系统设计，聊聊短期记忆、长期记忆、知识图谱三种方案各自适合什么场景。

关注我，James 的成长日记，持续分享干货，帮你在 AI 时代少走弯路。

AI Agent 修炼路 · 目录 ≡

<| 上一篇
Tool-Calling Agent 从零实现：
LangChain.js 实战

下一篇 >
Agent 记忆系统设计：短期、长期到知识图谱

作者提示: 个人观点，仅供参考

